

# Exploring TCP vs. UDP and Application Protocols in Mininet

Juan David Guarnizo Hernandez, PhD  
juan.guarnizo@tec.mx

**TC2006B: Device Communications**  
Tecnológico de Monterrey  
FJ2026

## Goals

By the end of this exercise you will:

1. Deploy a multi-host network topology using Mininet.
2. Deploy echo servers on TCP and UDP on a simulated host.
3. Use a Python client on another host to test both protocols.
4. Use traceroute and Censys to explore network paths across a NAT.
5. Capture and analyze traffic using `tcpdump` inside Mininet namespaces.
6. Compare HTTP versions over TCP and UDP/QUIC (using Docker for HTTP/3).
7. Explore DNS queries over UDP/TCP.

## 0) Pre-requirements & Environment Setup

### Note on Environment (Ubuntu 26.04)

This lab is designed to run natively on **Ubuntu 26.04**. We will use **Mininet** to create an isolated network of multiple hosts (`h1`, `h2`, etc.) connected to a switch.

- You will use `xterm` to open separate terminal windows for different Mininet hosts.
- Commands meant for a specific host must be typed into that host's `xterm` window.
- **Note:** As standard `apt` repositories do not yet compile `curl` with HTTP/3 support, we will use Docker for the HTTP/3 section.

### 0.1) Install Dependencies & Docker

Run the following commands in your main Ubuntu VM terminal to install the required networking tools:

```
1 sudo apt update
2 sudo apt install -y mininet tcpdump traceroute wireshark socat python3 curl dnsutils
  xterm
```

Next, install Docker. The default Ubuntu package (`docker.io`) is perfectly sufficient for our HTTP/3 tests:

```
1 sudo apt install -y docker.io
```

*Note on Docker Permissions:* By default, Docker requires root privileges (which is why we use `sudo docker` later in the lab). If you wish to run Docker without `sudo`, you can add your current VM user to the `docker` group:

```
1 sudo usermod -aG docker $USER
2 newgrp docker
```

(You may need to completely log out and log back into your Ubuntu VM for the group changes to take full effect globally).

## 0.2) Start the Mininet Topology

We will create a linear topology with 3 hosts and a NAT router so our hosts can access the external Internet (required for HTTP and traceroute). In your main terminal, run:

```
1 sudo mn --topo linear,3 --nat
```

Once the Mininet CLI (mininet>) starts, open terminal windows for hosts h1 and h2 by typing:

```
1 mininet> xterm h1 h1 h2
```

*Note: We opened two windows for h1 (one for running scripts, one for tcpdump) and one window for h2 (for our servers).*

---

## 1) Echo Client in Python

To understand TCP vs UDP, we will use a Python client on host h1 that sends a message to an echo server running on host h2 (IP: 10.0.0.2).

### Code

In one of the **h1 xterm windows**, create a file named `echo_client.py` with the following code:

```
1 import socket
2
3 def main():
4     # Mininet h2's default IP address
5     host = '10.0.0.2'
6     port = 3000
7
8     print("---- UDP Client ----")
9     with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
10         s.sendto(b'hello world to UDP server', (host, port))
11         data, addr = s.recvfrom(1024)
12         print('Received (UDP):', data.decode())
13
14     print("---- TCP Client ----")
15     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
16         s.connect((host, port))
17         s.sendall(b'hello world to TCP server')
18         data = s.recv(1024)
19         print('Received (TCP):', data.decode())
20
21 if __name__ == '__main__':
22     main()
```

## 2) Deploying Echo Servers with Socat

An echo server returns the same data it receives. We will deploy two servers (TCP and UDP) on host h2.

## Start the Servers on h2

In the **h2 xterm window**, run the TCP server in the background, followed by the UDP server:

```
1 socat TCP4-LISTEN:3000,fork EXEC:/bin/cat &
2 socat UDP4-LISTEN:3000,fork EXEC:/bin/cat &
```

## Capture Traffic and Run Client on h1

1. In the **first h1 xterm window**, start capturing traffic. Because we are inside the h1 namespace, we can safely listen on any interface:

```
1 sudo tcpdump -i any port 3000 -w echo_traffic.pcap
```

2. In the **second h1 xterm window**, execute your client:

```
1 python3 echo_client.py
```

3. Once the client finishes, go back to the first h1 window and stop the capture by pressing `Ctrl+C`.
- 

## 3) Exploring Network Paths with Traceroute

Because we started Mininet with the `--nat` flag, h1 can route packets out to the real Internet.

### Steps

1. **Find Target Servers with Censys:**

- Navigate to <https://search.censys.io/> on your host machine's web browser.
- Find web servers in a specific country (e.g., `services.service_name: HTTP` and `location.country: DE`).
- Pick two different public IP addresses.

2. **Trace the Route from h1:**

- In the **first h1 xterm window**, start a packet capture for ICMP or UDP:

```
1 sudo tcpdump -i any "icmp or udp" -w traceroute_traffic.pcap
```

- In the **second h1 xterm window**, trace the route to the IPs:

```
1 traceroute -n <IP_ADDRESS_1>
2 traceroute -n <IP_ADDRESS_2>
```

- Stop the `tcpdump` capture (`Ctrl+C`).
- 

## 4) Capturing and Analyzing HTTP Traffic

Since the default `curl` in Ubuntu 26.04 does not yet support HTTP/3 natively without compiling from source, we will use a Docker container. To avoid complex Docker-in-Mininet routing, we will execute this section directly on your **main Ubuntu host**.

## Steps

1. Open a **new main Ubuntu terminal window** (not an xterm) and start capturing traffic:

```
1 sudo tcpdump -i any port 80 or port 443 or udp port 443 -w http_traffic.pcap
```

2. In **another main terminal window**, run the following commands one by one:

```
1 curl -v http://example.com --http1.0 --header "Connection: close"
2 curl -v http://example.com --http1.1
3 docker run --rm alpine/curl-http3 curl -vk http://example.com --http3-only
4 docker run --rm alpine/curl-http3 curl -vk https://example.com --http3-only
```

3. Stop the `tcpdump` capture (Ctrl+C). You can open `http_traffic.pcap` in Wireshark on your base host later for analysis.
- 

## 5) Exploring DNS Queries

### Steps

1. In the **first h1 xterm window**, start capturing port 53 traffic:

```
1 tcpdump -i any port 53 -w dns_traffic.pcap
```

2. In the **second h1 xterm window**, generate DNS queries:

#### Using dig

```
1 dig example.com
2 dig A example.com      # Query IPv4 address records
3 dig MX example.com     # Query mail exchange records
4 dig AAAA example.com   # Query IPv6 address records
```

#### Using nslookup

```
1 nslookup example.com
2 nslookup -q=a example.com
3 nslookup -vc -q=a example.com # Forces TCP
```

Stop the capture (Ctrl+C) when finished. Once done with the lab, you can type `exit` in the Mininet CLI to tear down the topology.

---

## 6) Analysis and Reflection

Answer the following questions based on your observations by opening your `.pcap` files in Wireshark.

### Echo Client and Server (TCP vs UDP)

- Filter your capture by port 3000. How does the TCP 3-way handshake appear in your capture? What key differences do you observe between how the TCP message and the UDP message are transmitted on the wire?
- Run the client multiple times. Do you notice any difference in reliability or overhead between UDP and TCP within the Mininet environment?

## Network Paths (Traceroute)

- Look at the first hop in your traceroute. What IP address does it belong to, and why is it there? (Hint: Think about the `--nat` flag used in Mininet).
- Compare the paths to the two servers. Do they share the same hops initially? At which hop number do the paths diverge?
- What protocol does your version of traceroute use (ICMP or UDP)? Describe the specific messages you see in Wireshark.

## HTTP Traffic Analysis

- What differences do you observe in the TCP handshakes for HTTP/1.0 and HTTP/1.1?
- How is HTTP/3 different from HTTP/1.x in terms of transport protocol?
- One of the `docker run ... --http3-only` commands likely failed or behaved differently. Can you infer which one and why? (Hint: Does HTTP/3 work over plain unencrypted connections?)
- Which protocol is used for secure communications (`https://`), and does it only work for HTTP?

## DNS Query Analysis

- Identify whether each query used UDP or TCP.
- Compare the size of a typical DNS query and its response.
- What is the overhead (in packets and total bytes) introduced by forcing `nslookup` to use TCP with the `-vc` option compared to UDP?
- Where in the DNS response can you find how long the records should be cached? (Hint: Look for the TTL).